CharacterTextSplitter — Complete Theory, Parameters, Methods & Practical Programs

1. What Is CharacterTextSplitter?

Definition

CharacterTextSplitter is a LangChain class that splits text using a specified separator and combines the resulting pieces into chunks according to configured chunk size and overlap rules.

Important:

CharacterTextSplitter

↓

Find Separator

↓

Split Text

↓

Merge Smaller Pieces

↓

Create Final Chunks

It does **not always cut text after exactly 100, 200, or any fixed number of characters**.

It first looks for the configured separator and then creates chunks from those pieces.

2. Main Advantages

CharacterTextSplitter provides:

1. Reusable text-splitting functionality.
2. Configurable separators.
3. chunk_size support.
4. chunk_overlap support.
5. Direct support for LangChain Document objects.
6. Ability to preserve metadata.

3. Python split() vs CharacterTextSplitter

Python already provides:

text.split(",")

So why do we need a LangChain splitter?

Suppose:

text = "Apple,Banana,Mango"

Normal Python:

parts = text.split(",")

Result:

```
[
   "Apple",
```

"Banana",

"Mango"

]

Python simply divides the string using the delimiter.

4. What Extra Work Does LangChain Do?

With LangChain text splitters, we may want to:

- Split text using separators.
- Combine smaller pieces into larger chunks.
- Control approximate chunk size.
- Maintain overlap.
- Split Document objects.
- Preserve Document metadata.

So:

Python split()

↓

Only Split

Whereas:

CharacterTextSplitter

↓

Split

+

Merge

+

Chunk Size

+

Overlap

+

Documents

+

Metadata

5. Python str.split() vs CharacterTextSplitter

Python str.split()	**CharacterTextSplitter**
Basic string method	LangChain class
Splits using a delimiter	Splits and merges text
No chunk-size configuration	Supports chunk_size
No overlap configuration	Supports chunk_overlap
Returns list of strings	Can return strings or Documents

Does not manage metadata	Supports Document metadata

6. How Does CharacterTextSplitter Work Internally?

Suppose:

text = "AAA|BBB|CCC|DDD"

Configuration:

separator = "|"

chunk_size = 7

chunk_overlap = 0

7. Step 1 — Identify the Separator

Separator:

|

The splitter checks where this separator occurs.

8. Step 2 — Divide into Smaller Pieces

Conceptually:

AAA

BBB

CCC

DDD

At this point, these are smaller split pieces.

9. Step 3 — Merge Pieces into Final Chunks

The splitter tries to combine smaller pieces without crossing the target chunk size.

With chunk_size=7:

AAA|BBB

length:

7

Next:

CCC|DDD

length:

7

So final result:

Chunk 1: AAA|BBB

Chunk 2: CCC|DDD

Important:

Splitting and merging are two separate parts of the process.

10. Master Working Formula

Original Text

↓

Separator

↓

Smaller Pieces

↓

Merge Pieces

↓

chunk_size

↓

chunk_overlap

↓

Final Chunks

11. Import

```
from langchain_text_splitters import (
   CharacterTextSplitter
)
```

12. Create Splitter Object

```
splitter = CharacterTextSplitter(
    separator="|",
    chunk_size=7,
    chunk_overlap=0
)
```

13. Split Text

```
chunks = splitter.split_text(
    text
)
```

Here:

CharacterTextSplitter(...)

↓

Creates Splitter Object

split_text(text)

↓

Performs Splitting

chunks

↓

List of Strings

14. Important Constructor Parameters

Parameter	Default	Purpose
separator	"\n\n"	Splitting boundary
chunk_size	4000	Target maximum chunk length
chunk_overlap	200	Target overlap
length_function	len	Measures text length
keep_separator	False	Controls separator retention
is_separator_regex	False	Treat separator as regex when enabled
strip_whitespace	True	Removes leading/trailing whitespace
add_start_index	False	Adds start position to Document metadata

15. Understanding separator

Example:

separator="|"

means:

Look for |

↓

Use It as a Possible

Splitting Boundary

Default:

separator="\n\n"

which commonly represents a paragraph boundary.

16. Understanding chunk_size

Example:

chunk_size=7

means:

Try to Create Chunks

Around the Target

Maximum Length of 7

Important:

It is not always a strict fixed-width cut.

17. Understanding chunk_overlap

Example:

chunk_overlap=3

means:

Try to Repeat Some Text

Between Nearby Chunks

Purpose:

Preserve Context

Across Chunk Boundaries

18. Understanding length_function

Default:

length_function=len

This determines how chunk length is measured.

With:

len

the splitter measures:

Python String Length

19. Understanding keep_separator

Default:

keep_separator=False

It controls how separators are retained during splitting and merging.

The source also notes support for:

keep_separator="start"

and:

keep_separator="end"

20. Understanding is_separator_regex

Default:

is_separator_regex=False

This means the separator is treated as:

Literal Text

If:

is_separator_regex=True

the separator is interpreted as:

Regular Expression

21. Understanding strip_whitespace

Default:

strip_whitespace=True

This removes leading and trailing whitespace from final merged chunks.

Example:

" Python "

becomes:

"Python"

22. Understanding add_start_index

Example:

add_start_index=True

When splitting Documents, this can add:

"start_index"

to chunk metadata.

It tells us where the chunk starts in the original text.

23. Program 1 — First CharacterTextSplitter Program

```
from langchain_text_splitters import (
    CharacterTextSplitter
)



# ========================================
# ORIGINAL TEXT
# ========================================

text = "AAA|BBB|CCC|DDD"



# ========================================
# CREATE TEXT SPLITTER
# ========================================

splitter = CharacterTextSplitter(
    separator="|",
    chunk_size=7,
```

```
    chunk_overlap=0
)


# ========================================
# SPLIT TEXT
# ========================================

chunks = splitter.split_text(
    text
)


# ========================================
# DISPLAY RESULTS
# ========================================

print(
    "ORIGINAL TEXT:",
    text
```

```
)

print(
    "TOTAL CHUNKS:",
    len(chunks)
)

print(
    "RETURN TYPE:",
    type(chunks)
)


for index, chunk in enumerate(
    chunks,
    start=1
):

    print(
        f"Chunk {index}: {chunk} "
```

```
        f"| Length: {len(chunk)}"
    )
```

24. Program 1 Output

```
ORIGINAL TEXT: AAA|BBB|CCC|DDD

TOTAL CHUNKS: 2

RETURN TYPE: <class 'list'>

Chunk 1: AAA|BBB | Length: 7

Chunk 2: CCC|DDD | Length: 7
```

25. Program 1 Flow

```
AAA|BBB|CCC|DDD
      ↓
separator="|"
      ↓
AAA
```

BBB

CCC

DDD

↓

chunk_size=7

↓

AAA|BBB

CCC|DDD

26. Program 2 — Understanding the Default Separator

Default separator:

"\n\n"

This commonly means:

Paragraph Boundary

27. Program 2

```
from langchain_text_splitters import (
    CharacterTextSplitter
)


text = (
    "Python is easy.\n\n"
    "Java is powerful.\n\n"
    "LangChain builds AI apps."
)


splitter = CharacterTextSplitter(
    chunk_size=20,
    chunk_overlap=0
)


chunks = splitter.split_text(
```

```
    text
)



print(
    "TOTAL CHUNKS:",
    len(chunks)
)



for index, chunk in enumerate(
    chunks,
    start=1
):

    print(
        f"\nCHUNK {index}"
    )

    print(
```

```
        repr(chunk)
    )

    print(
        "LENGTH:",
        len(chunk)
    )
```

28. Program 2 Output

```
TOTAL CHUNKS: 3

CHUNK 1
'Python is easy.'
LENGTH: 15

CHUNK 2
'Java is powerful.'
LENGTH: 17

CHUNK 3
```

'LangChain builds AI apps.'

LENGTH: 25

29. Very Important Observation

Configured:

chunk_size=20

But one chunk has:

25 characters

Why?

Because the selected splitter may not be able to further divide that piece using its configured separator.

Therefore:

Target chunk size does not always guarantee a strict maximum when the selected separator cannot split a large piece further.

30. Program 3 — Most Important Limitation: Missing Separator

Suppose:

text = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

Splitter:

```
splitter = CharacterTextSplitter(
    separator="\n\n",
    chunk_size=5,
    chunk_overlap=0
)
```

But the text contains no:

"\n\n"

separator.

31. Program 3

```
from langchain_text_splitters import (
    CharacterTextSplitter
)



text = (
    "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
)
```

```
splitter = CharacterTextSplitter(
    separator="\n\n",
    chunk_size=5,
    chunk_overlap=0
)


chunks = splitter.split_text(
    text
)


print(
    "ORIGINAL LENGTH:",
    len(text)
)


print(
    "CONFIGURED CHUNK SIZE:",
```

```
    5
)



print(
    "TOTAL CHUNKS:",
    len(chunks)
)



for index, chunk in enumerate(
    chunks,
    start=1
):

    print(
        f"Chunk {index}: {chunk}"
    )

    print(
```

```
        "Length:",
        len(chunk)
    )
```

32. Program 3 Output

ORIGINAL LENGTH: 26

CONFIGURED CHUNK SIZE: 5

TOTAL CHUNKS: 1

Chunk 1:

ABCDEFGHIJKLMNOPQRSTUVWXYZ

Length: 26

33. Why Did chunk_size=5 Not Work Here?

Because the configured separator:

"\n\n"

does not exist in the text.

So the splitter cannot create smaller separator-based pieces.

This proves:

CharacterTextSplitter is separator-based, not a strict fixed-width slicing algorithm.

34. When Should We Use RecursiveCharacterTextSplitter?

Suppose we want the splitter to try:

Paragraph

↓

Line

↓

Space

↓

Character

Then the source recommends:

RecursiveCharacterTextSplitter

35. CharacterTextSplitter vs RecursiveCharacterTextSplitter

CharacterTextSplitter

↓

One Configured Separator

RecursiveCharacterTextSplitter

↓

Tries Multiple Separators

in Sequence

36. Program 4 — Splitting Using Space Separator

```
from langchain_text_splitters import (
   CharacterTextSplitter
)



text = (
   "Apple Banana Mango Orange"
)
```

```
splitter = CharacterTextSplitter(
    separator=" ",
    chunk_size=12,
    chunk_overlap=0
)


chunks = splitter.split_text(
    text
)


print(
    "TOTAL CHUNKS:",
    len(chunks)
)


for index, chunk in enumerate(
    chunks,
```

```
    start=1
):

    print(
        f"Chunk {index}: "
        f"{repr(chunk)}"
    )

    print(
        "Length:",
        len(chunk)
    )
```

37. Program 4 Output

```
TOTAL CHUNKS: 2

Chunk 1:
'Apple Banana'
Length: 12
```

Chunk 2:

'Mango Orange'

Length: 12

38. Program 4 Internal Flow

Original text:

Apple Banana Mango Orange

Separator:

Space

Initial pieces:

Apple

Banana

Mango

Orange

Then they are merged according to:

chunk_size=12

Final chunks:

Apple Banana

Mango Orange

39. Program 5 — Chunk Overlap

Configuration:

separator="|"

chunk_size=7

chunk_overlap=3

40. Program 5

```
from langchain_text_splitters import (
    CharacterTextSplitter
)



text = "AAA|BBB|CCC|DDD"



splitter = CharacterTextSplitter(
    separator="|",
    chunk_size=7,
    chunk_overlap=3
)
```

```
chunks = splitter.split_text(
    text
)


for index, chunk in enumerate(
    chunks,
    start=1
):

    print(
        f"Chunk {index}: {chunk}"
    )
```

41. Program 5 Output

Chunk 1: AAA|BBB

Chunk 2: BBB|CCC

Chunk 3: CCC|DDD

42. Understanding Overlap Here

Observe:

Chunk 1:

AAA|BBB

and:

Chunk 2:

BBB|CCC

Repeated piece:

BBB

Then:

Chunk 2:

BBB|CCC

and:

Chunk 3:

CCC|DDD

Repeated piece:

CCC

So:

Overlap

↓

Repeated Content

Between Nearby Chunks

43. Program 6 — keep_separator=False vs True

Text:

text = "AAA|BBB|CCC"

44. Approach 1 — keep_separator=False

```
splitter1 = CharacterTextSplitter(
    separator="|",
    chunk_size=3,
    chunk_overlap=0,
    keep_separator=False
```

)

Output:

'AAA'

'BBB'

'CCC'

45. Approach 2 — keep_separator=True

```
splitter2 = CharacterTextSplitter(
    separator="|",
    chunk_size=3,
    chunk_overlap=0,
    keep_separator=True
)
```

Output shown in the source:

'AAA'

'|BBB'

'|CCC'

46. Important Observation About keep_separator=True

The separator appears at the beginning of later chunks.

Example:

|BBB

|CCC

This makes the boundary visible in the output.

47. str() vs repr()

str()	**repr()**
Human-readable representation	Developer-friendly representation
Focuses on readability	Useful for debugging
Special characters may appear as formatting	Escape sequences such as \n remain visible

48. Why Use repr()?

Suppose:

text = "Python\nJava"

Using:

print(text)

shows:

Python

Java

But:

print(

repr(text)

)

shows:

'Python\nJava'

So hidden formatting characters become visible.

Important:

repr() does not change the original string.

49. repr() Memory Formula

str()

↓

READ

repr()

↓

INSPECT / DEBUG

50. Can We Keep Separator at the End?

Yes.

Use:

keep_separator="end"

Example:

```
splitter = CharacterTextSplitter(
    separator="|",
    chunk_size=4,
    chunk_overlap=0,
    keep_separator="end"
)
```

Output:

'AAA|'

'BBB|'

'CCC'

51. Important Clarification About keep_separator=False

Do not misunderstand:

keep_separator=False

It does **not necessarily mean separators can never appear in final merged chunks**.

The source notes that separators may be reinserted while smaller pieces are merged.

52. Program 7 — split_documents() and Metadata

Suppose we have:

```
Document(
   page_content=
      "AAA|BBB|CCC|DDD",

   metadata={
      "source":
         "company_policy.txt",

      "department":
         "HR"
   }
```

)

We want to split this Document while preserving metadata.

53. Program 7

```
from langchain_core.documents import (
   Document
)

from langchain_text_splitters import (
   CharacterTextSplitter
)



# ========================================
# CREATE DOCUMENT
# ========================================

document = Document(
   page_content=
      "AAA|BBB|CCC|DDD",
```

```
    metadata={
        "source":
            "company_policy.txt",

        "department":
            "HR"
    }
)



# ========================================
# CREATE SPLITTER
# ========================================

splitter = CharacterTextSplitter(
    separator="|",
    chunk_size=7,
    chunk_overlap=0
)
```

```
# ========================================
# SPLIT DOCUMENT
# ========================================

chunks = splitter.split_documents(
    [
        document
    ]
)



# ========================================
# DISPLAY RESULTS
# ========================================

print(
    "TOTAL CHUNKS:",
    len(chunks)
```

```
)


for index, chunk in enumerate(
    chunks,
    start=1
):

    print(
        f"\nCHUNK {index}"
    )


    print(
        "CONTENT:",
        chunk.page_content
    )


    print(
```

```
    "METADATA:",
    chunk.metadata
)


print(
    "TYPE:",
    type(chunk).__name__
)
```

54. Important Difference

split_text()

↓

Returns Strings

split_documents()

↓

Returns Document Objects

55. split_text() Return Type

Input:

"AAA|BBB|CCC|DDD"

Output conceptually:

[

"AAA|BBB",

"CCC|DDD"

]

Return type:

list[str]

56. split_documents() Return Type

Input:

[

Document(...)

]

Output:

[

Document(...),

Document(...)

]

Return type:

list[Document]

57. Why Is Metadata Preservation Important?

Suppose original metadata is:

```
{
   "source":
      "company_policy.txt",

   "department":
      "HR"
}
```

After splitting, chunks can continue carrying this information.

This is useful for:

Source Tracking

Filtering

Retrieval

RAG

Citation

Department-Based Search

58. Program 8 — Understanding add_start_index

Student question:

After splitting a Document, how can we find the original starting position of every chunk?

LangChain provides:

add_start_index=True

59. Program 8

```
from langchain_core.documents import (
   Document
)

from langchain_text_splitters import (
   CharacterTextSplitter
)



document = Document(
```

```
    page_content=
        "AAA|BBB|CCC|DDD",

    metadata={
        "source":
            "demo.txt"
    }
)


splitter = CharacterTextSplitter(
    separator="|",
    chunk_size=7,
    chunk_overlap=0,
    add_start_index=True
)


chunks = splitter.split_documents(
    [
```

```
        document
    ]
)


for index, chunk in enumerate(
    chunks,
    start=1
):

    print(
        f"\nCHUNK {index}"
    )


    print(
        "CONTENT:",
        chunk.page_content
    )
```

```
print(
    "START INDEX:",
    chunk.metadata[
        "start_index"
    ]
)


print(
    "SOURCE:",
    chunk.metadata[
        "source"
    ]
)
```

60. What Is start_index?

It represents:

Starting Character Position

of the Chunk

in the Original Text

For example:

Original:

AAA|BBB|CCC|DDD

If a chunk starts from the beginning:

start_index = 0

Later chunks may have a different starting position.

61. Why Is add_start_index Useful?

It helps us know:

Where Did This Chunk

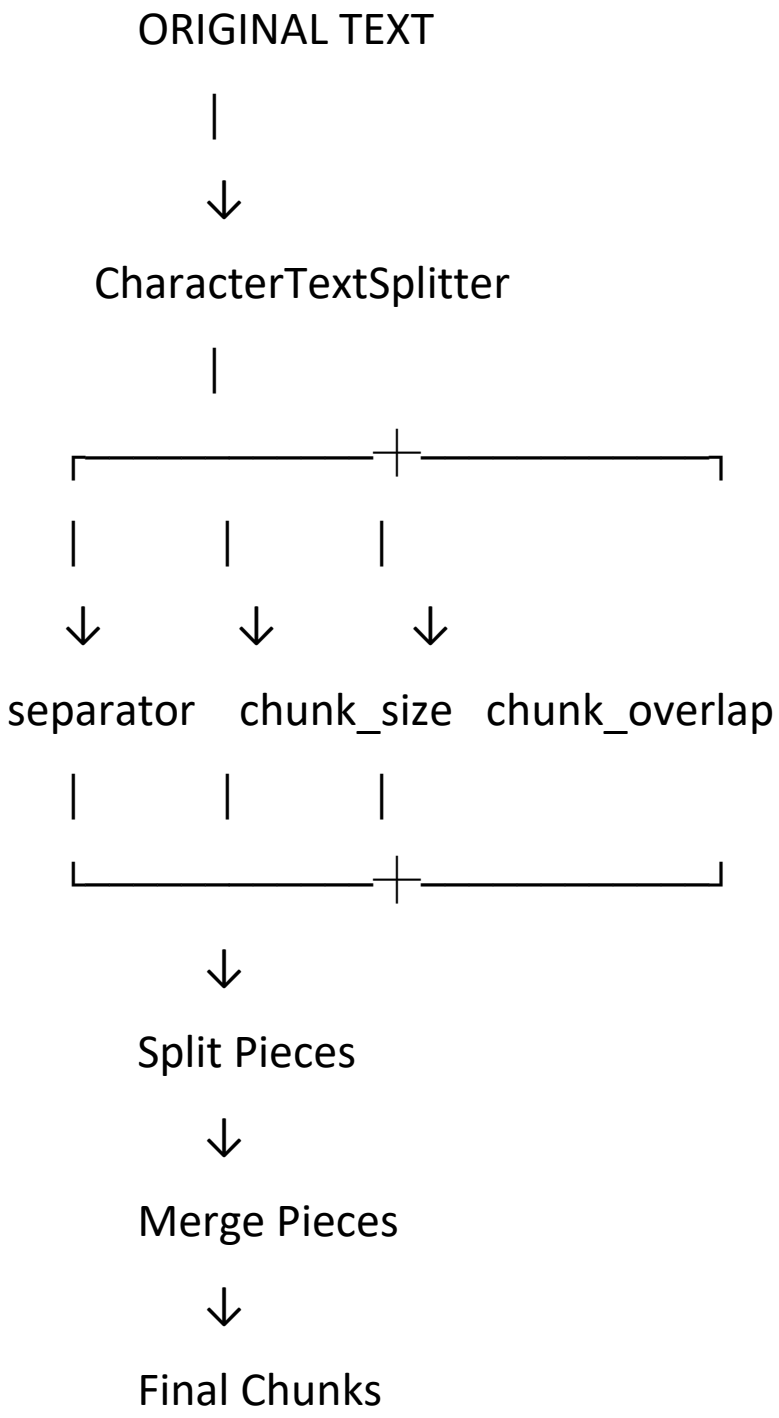
Come From in the

Original Text?

Useful for:

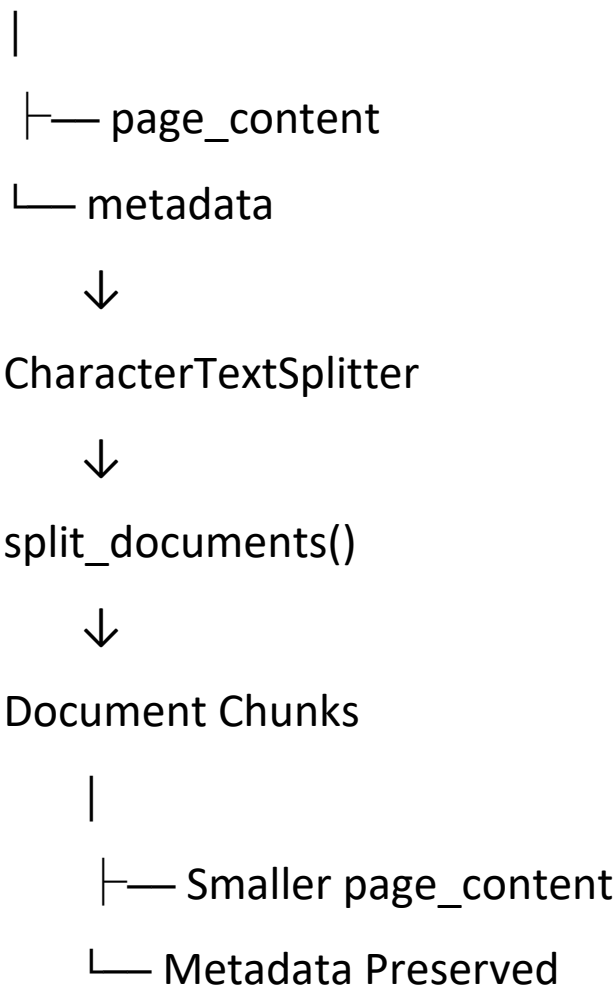
- Traceability
- Debugging
- Highlighting source text
- Retrieval systems
- RAG applications

62. Complete CharacterTextSplitter Architecture



63. When Splitting Documents

Original Document



64. Most Important Limitation

CharacterTextSplitter depends strongly on its separator.

Suppose:

separator="\n\n"

but text has no:

\n\n

Then even if:

chunk_size=5

the splitter may still return a much larger piece.

So:

CharacterTextSplitter is separator-based, not strict character slicing.

65. Common Mistake — Thinking chunk_size Means Exact Cut

Wrong:

chunk_size=5

↓

Always Cut Every 5 Characters

Correct:

separator

↓

Split Pieces

↓

chunk_size

↓

Guide Merging

66. Common Mistake — Thinking Separator Creates Final Chunks Directly

Not always.

Example:

AAA|BBB|CCC|DDD

First:

AAA

BBB

CCC

DDD

Then merging may create:

AAA|BBB

CCC|DDD

So:

Splitting

≠

Final Chunk Formation

67. Common Mistake — Thinking Missing Separator Will Force Character Splitting

Not with CharacterTextSplitter.

If the separator is absent:

No Split Point Found

↓

Large Piece May Remain

For multiple fallback separators, use:

RecursiveCharacterTextSplitter

68. Common Mistake — Thinking keep_separator=False Means Separator Can Never Appear

Wrong.

The source notes that separators can be reinserted when pieces are merged.

So:

keep_separator

↓

Controls Split Behaviour

but final merging also affects the resulting chunk text.

69. Common Mistake — Thinking repr() Changes the String

Wrong.

repr(text)

only gives a representation useful for inspection.

Original text remains unchanged.

70. Common Mistake — Confusing split_text() and split_documents()

split_text()

↓

String Input

↓

String Chunks

split_documents()

↓

Document Input

↓

Document Chunks

71. Important Program Summary

Program 1

Custom Separator

↓

Split + Merge

Program 2

Default Separator

↓

"\n\n"

Program 3

Missing Separator

↓

Important Limitation

Program 4

Space Separator

↓

Word-Based Pieces

Program 5

Chunk Overlap

↓

Repeated Nearby Content

Program 6

keep_separator

↓

False / True / End

Program 7

split_documents()

↓

Document Chunks

+

Metadata

Program 8

add_start_index=True

↓

Original Chunk Position

72. Important Parameter Revision

separator

↓

WHERE to split?

chunk_size

↓

HOW LARGE should chunks be?

chunk_overlap

↓

HOW MUCH should repeat?

length_function

↓

HOW is length measured?

keep_separator

↓

HOW is separator retained?

is_separator_regex

↓

Literal or Regex?

strip_whitespace

↓

Remove surrounding spaces?

add_start_index

↓

Store original position?

73. CharacterTextSplitter Master Formula

TEXT

↓

SEPARATOR

↓

SMALLER PIECES

↓

MERGING

↓

CHUNK SIZE

↓

CHUNK OVERLAP

↓

FINAL CHUNKS

74. Most Important Learning Formula

Separator decides where splitting can happen.

Chunk size controls how much text should be grouped into a chunk.

Chunk overlap controls how much nearby content should repeat.

CharacterTextSplitter is separator-based, not strict fixed-width slicing.

75. Final One-Line Conclusion

CharacterTextSplitter first divides text using a configured separator and then merges those pieces into chunks according to chunk-size and overlap rules; it can also split LangChain Documents while preserving metadata and optionally recording each chunk's original start position.